Technical Assessment — QA Team Lead

Format: Take-home · Hands-on · 2–3 hours
Submission: GitHub repo (preferred) or zip. Include a README with setup instructions and notes on what you'd do differently with more time.
Language: Your choice. Use whatever you're most productive in.
Sections: 2 sections. Section 1 requires working code. Section 2 is a short written answer.

> We value working code over comprehensive coverage. Submit what you finish within the time box — don't over-engineer.

Section 1 — Test Automation

Time guidance: 60–75 min · Code required

Application under test: https://www.saucedemo.com (public demo, no sign-up needed)
Credentials: Username: `standard_user` · Password: `secret_sauce`

Task 1.1 — Automate the purchase flow

Scenario
Using Selenium, Playwright, or Cypress, automate the following:

1. Log in with valid credentials
2. Add the two cheapest items to the cart
3. Proceed to checkout and complete the order
4. Verify the order confirmation page appears

Deliver
Working test code in a repo. Must run from a clean checkout with a single command (e.g. `pytest`, `npm test`). Include a README with the run command.

> We care about structure and reliability, not breadth. Two solid, stable tests beat five flaky ones.

Task 1.2 — Add one negative test

Scenario
Test the login flow with invalid credentials (wrong password). Verify the correct error message appears.

Deliver
One additional test case in the same framework. Assertion must be on the error message text — not just that the element exists.

Section 2 — API Testing

Time guidance: 20–25 min · Written only

Task 2.1 — Test case design

Scenario
Given this endpoint:

```
POST /api/cart/add
Body: { "product_id": integer, "quantity": integer }
Auth: Bearer token required
```

List 6–8 test cases you would write. For each: scenario name, input, expected HTTP status.

Deliver
A simple table or list. No need to write code.

> Include at least one security-related case and one boundary case.

What we're looking for

Dimension	What we are looking for
Code that actually runs	Section 1 should work from a clean clone. Broken env setup is a red flag.
Structure over cleverness	How you organise tests (POM, fixtures, helpers) matters more than clever one-liners.
Honest trade-offs	If you ran out of time, say what you'd add next and why. That's more valuable than silence
Stability over volume	Three reliable, well-structured tests beat ten that pass only sometimes.

Questions before you start? Reach out to the hiring team. Good luck.